DECODE & DETECT HACKATHON · PROJECT REPORT

Ledgerwatch

Finding money-laundering patterns in XRP Ledger payments

Team Code Gladiators

1. What the project does

The hackathon supplies a week of **synthetic payment activity on a fictional XRP Ledger**. Hidden among ordinary payments are **planted money-laundering patterns**: fan-outs, fan-ins, relay chains, cycles, layering and more. Ledgerwatch gives **every transaction a suspicion score from 0 to 1** so that suspicious payments rise to the top of an investigator's review list.

Part	Folder	What it does
Detection pipeline	src/	Joins the raw ledger files into one row per payment, builds ~190 behavioural features (activity bursts, fast forwarding, chains, cycles, one-off wallet pairs) and trains a LightGBM model.
Investigation dashboard	api/ + frontend/	A FastAPI backend and a Next.js website to browse the scored payments, see why each one was flagged, explore the money-flow graph and compare models.

In one line: raw payments → join and value in USD → behavioural features → LightGBM → a 0-1 score per payment → a dashboard that ranks and explains them.

2. How it works

• **Load and join** (`tx_data.py`): transactions, transfer legs, blocks and hourly market rates become one row per payment, valued in USD.
• **Features** (`tx_features.py`, `tx_extra.py`, `tx_patterns.py`): numbers that describe behaviour around each payment, built the same way for training and test data.
• **Train and score** (`run_model.py`): validates the model, trains it on the labelled data, scores the 40,101 test payments and checks the output CSV.
• **Dashboard** (`api/`, `frontend/`): reads the saved results and shows them in the browser.

3. Model

LightGBM v2 (LightGBM + ratio, activity and pattern features): it gave the best validation score of every model we tried (XGBoost, CatBoost, logistic regression, Isolation Forest), runs in minutes on a laptop CPU and can show which features drove each score.

4. How to run

Prerequisites: Python 3.11, Node.js 20+, and the competition data placed like this (not included in the repository - the organisers' data may not be redistributed):

```
XPRL_HACKATHON/
  decode-and-detect-training-data/train/   transactions.parquet  transfers.parquet  blocks.parquet
                                           market_rates.csv  labels.csv
  decode-and-detect-test-data/             same files, without labels.csv
```

Step 1 - install

```
python -m pip install -r requirements.txt
python -m pip install xgboost catboost      # imported by the shared model module
cd frontend ; npm install ; cd ..
```

Step 2 - train the model and score the test data

```
python src/run_model.py --model lightgbm --extra ratios activity patterns --dashboard
#  -> outputs/models/lightgbm_activity_patterns_ratios/submission.csv  (40,101 rows)
```

Step 3 - start the dashboard. One click on Windows: `.\run_dashboard.ps1` (builds missing model files, then opens both servers; edit `$py` at the top if your Python is not `F:\ana\envs\xrpl_env`). Or by hand:

```
# terminal 1 - API on :8000
$env:TXMODEL_RUN = "lightgbm_activity_patterns_ratios"
python -m uvicorn api.main:app --port 8000

# terminal 2 - website on :3000
cd frontend ; npm run dev
```

Open **http://localhost:3000** for the dashboard and **http://127.0.0.1:8000/docs** for the API.

5. Dashboard pages

Page	What you see
01 Dashboard	headline metrics, score distribution, activity over time, top suspicious payments
02 Transactions	ranked, filterable payments; detail view with legs, block, rate, wallet context and the top reasons behind the score
03 Network graph	wallet-to-wallet money flow around a payment or wallet
04 Models	every trained model side by side
05 Validation	validation folds, leakage checks, feature importance, submission checks

6. Project structure

Path	Role
src/tx_data.py	load and join the 5 files into one row per payment (checked joins, USD values)
src/tx_features.py	~134 behavioural features
src/tx_extra.py, src/tx_patterns.py	extra feature groups: ratios, activity, pattern structure and pace
src/competition_max.py	model definitions (LightGBM / XGBoost / CatBoost)
src/run_model.py	main entry point: validate → train → score test → check CSV → dashboard files
src/competition.py, src/compare_quick.py	shared helpers (metrics, leakage report), feature cache, quick model comparison
src/lgbm_v3.py	self-contained v3 pipeline that also writes the upload ZIP
src/package_submission.py	builds predictions.csv + source_code.zip for any run
src/ex1-ex4_*.py, xrpl_common.py	exercises on real XRPL Mainnet data via public JSON-RPC, with a ledger cache
src/make_sample_data.py ... detect.py	early account-level prototype: random forest + typology rules + Isolation Forest
api/	FastAPI backend: main.py (app, datasets, accounts) and v2.py (/api/v2 endpoints)
frontend/	Next.js dashboard: app/ (pages), components/ (charts, graph, layout), lib/ (API client)
outputs/	generated runs, submissions and caches (git-ignored)
docs/	this report
run_dashboard.ps1	one-click start on Windows
requirements.txt	Python dependencies (pinned)
SUBMISSION_README.md, THIRD_PARTY.md	README shipped in the upload ZIP; what we did not write ourselves
LICENSE	MIT licence

7. License

Ledgerwatch is released under the **MIT License**, © 2026 Code Gladiators.
https://github.com/jobin2201/XPRL_HACKATHON/blob/main/LICENSE
Repository: https://github.com/jobin2201/XPRL_HACKATHON